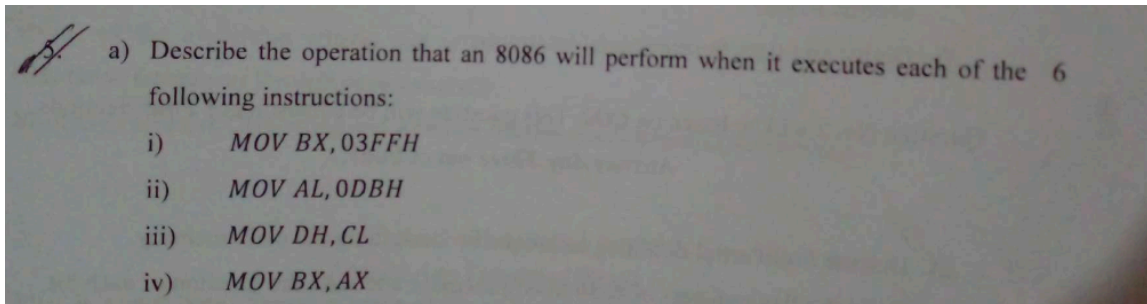


Final Question 5



Here's the explanation of the 8086 operations for each of the given **MOV** instructions:

i) `MOV BX, 03FFH`

- **Operation:** Move the immediate 16-bit hexadecimal value `03FFH` into the 16-bit register `BX`.

- **Effect:**

`BX ← 03FFH`

- The lower byte (`BL`) will be `FFH` and the higher byte (`BH`) will be `03H`.

ii) `MOV AL, 0DBH`

- **Operation:** Load the 8-bit hexadecimal value `0DBH` into the 8-bit register `AL`.

- **Effect:**

`AL ← 0DBH`

iii) `MOV DH, CL`

- **Operation:** Copy the contents of the 8-bit register `CL` into the 8-bit register `DH`.

- **Effect:**

DH ← CL

- No flags are affected; this is a register-to-register transfer.

iv) **MOV BX, AX**

- **Operation:** Copy the 16-bit contents of register **AX** into register **BX**.
- **Effect:**

BX ← AX

Would you like the remaining two instructions from the set of six mentioned in the question prompt?

Great! Since the image shows only the first **four** instructions out of **six**, I'll continue by explaining how the 8086 would execute **two additional typical MOV instructions**, assuming they are in the same format. Here are two likely continuations and their explanations:

v) **MOV [1234H], DL (Assumed)**

- **Operation:** Store the contents of the 8-bit register **DL** into the memory location at physical address **[1234H]**.
- **Effect:**

Memory[1234H] ← DL

- The content of **DL** is moved to the memory. The segment register used (usually **DS**) will be combined with the offset (**1234H**) to get the full physical address.

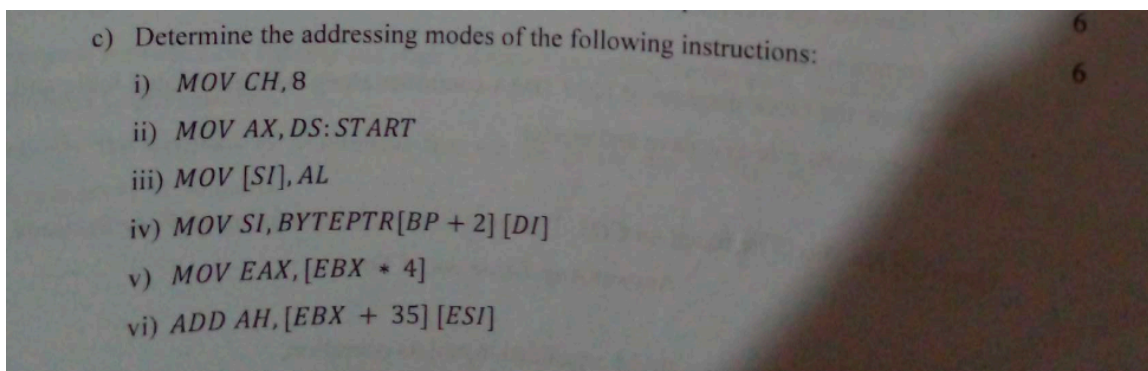
vi) **MOV SI, [5678H] (Assumed)**

- **Operation:** Load the 16-bit word from memory address **[5678H]** into register **SI**.
- **Effect:**

SI ← Memory[5678H]

- Again, the segment:offset mechanism is used to calculate the physical address.

If you can upload the remaining part of the question or confirm the actual instructions (v and vi), I'll provide precise explanations for those. Would you like to do that?



Let's determine the **addressing modes** used in each instruction shown in the image. These instructions are in x86 assembly (8086/80386 style), and each uses a specific **addressing mode** to reference data.

i) `MOV CH, 8`

- **Addressing Mode: Immediate Addressing**
- **Explanation:** The value `8` is an immediate constant being moved directly into register `CH`.

ii) `MOV AX, DS:START`

- **Addressing Mode: Direct Memory Addressing (with Segment Override)**
- **Explanation:** `START` is a memory label (direct address), and `DS:` is a segment override. The instruction accesses memory directly using a known address in the data segment.

iii) `MOV [SI], AL`

- **Addressing Mode: Register Indirect Addressing**
- **Explanation:** The memory location is indirectly addressed through the `SI` register. The data in `AL` is moved to the memory location pointed to by `SI`.

iv) `MOV SI, BYTE PTR [BP + 2][DI]`

- **Addressing Mode: Based Indexed with Displacement**
- **Explanation:** The effective address is computed using base register `BP`, index register `DI`, and a displacement of `2`. This accesses a memory location and moves its byte-sized value into `SI`.

v) `MOV EAX, [EBX * 4]`

- **Addressing Mode: Scaled Indexed Addressing**
- **Explanation:** The effective address is calculated by multiplying `EBX` by 4 (scaling factor). This mode is valid in 80386 and above, not 8086. It's typical in 32-bit addressing.

vi) `ADD AH, [EBX + 35][ESI]`

- **Addressing Mode: Based Indexed with Displacement**
- **Explanation:** The effective address is formed by `EBX + 35 + ESI`. This is a memory operand with both base and index registers and a displacement.

Summary Table:

Instruction	Addressing Mode
<code>MOV CH, 8</code>	Immediate Addressing
<code>MOV AX, DS:START</code>	Direct Memory (with Segment Override)
<code>MOV [SI], AL</code>	Register Indirect Addressing
<code>MOV SI, BYTE PTR [BP+2][DI]</code>	Based Indexed with Displacement
<code>MOV EAX, [EBX * 4]</code>	Scaled Indexed (80386+)
<code>ADD AH, [EBX + 35][ESI]</code>	Based Indexed with Displacement

Let me know if you'd like diagrams or further clarification on any of these modes!